```
       disjointSet = new
DisjointSet(graph.vertices);                          #1
       for each edge in
graph.edges
#2
         disjointSet.merge(edge.source,
edge.destination)                    #3
```

#1 Create a new disjoint set where each vertex of the graph is initially in a different partition
#2 Loop over each edge in the graph
#3 Merge the partitions to which source and destination vertices belong to

At the end, each partition of vertices in `disjointSet` will be a connected component.

It's worth noting that this algorithm **can NOT** be used for *directed graphs* and *strongly connected components*.

5.7.2 Graphs[14]: Kruskal Algorithm for Minimum Spanning Tree

A spanning tree for a connected undirected *graph* `G` is a tree whose nodes are the vertex of the graphs, and whose edges are a subset of `G`'s edges. If `G` is connected, then it certainly has at least one spanning tree, possibly many, if it also has cycles (see figure 5.12).